CNG KSP Feature Matrix

Every capability observed across shipping CNG Key Storage Providers, with this project's coverage and — where a gap exists — what closing it would take. Generated 2026-10-17.

104	74	10	20	71%	10	11
Capabilities tracked	Covered	Partial	Not covered	Fully covered	Startable here	Sized but blocked

How to read this

Status	Meaning
Covered	Implemented and exercised by the test suite.
Partial	Implemented, but reachable only through a non-standard identifier or with a stated caveat.
Not covered	Absent. The Gap column says what closing it would take.

Ev	Evidence for the market claim
A	Read from source code or an authoritative API reference that enumerates the provider's own algorithm list.
B	Stated in vendor or Microsoft documentation surfaced through search; primary page not read in full.
C	Drawn from HSM datasheets describing the device rather than its KSP. Treat as a lead, not a fact.

Effort on gaps: S = a few lines to a day · M = days · L = a substantial piece of work. **N/A** means no remedy is planned at all. **Effort is not the same question as whether a row can start**: TABLE-01 is S and needs a Windows machine. “Startable here” counts only sized rows whose blocker is a decision this project made; sized rows blocked outside this repository are counted beside it rather than folded in.

ID	Feature	Detail	Market support	Ev	Status	Gap — what closing it would take	Eff
PROVIDER INTERFACE							
`IFACE-01`	**GetKeyStorageInterface export**	Sole exported symbol; returns the KSP function table	All providers	A	**Covered**	—	—
`IFACE-02`	**OpenProvider / FreeProvider**	Provider handle lifecycle	All providers	A	**Covered**	—	—
`IFACE-03`	**GetProviderProperty**	Name / Version / ImplType	All providers	A	**Covered**	—	—

ID	Feature	Detail	Market support	Ev	Status	Gap — what closing it would take	Eff
IFACE-04	**SetProviderProperty**	NCRYPT_■PIN_■PROPERTY is implemented: the PIN is handed to the session layer, and because sessions open lazily a PIN set before the first cryptographic call is the one used to log in. Token selection is now writable too, by KSP_■TOKEN_■LABEL_■PROPERTY or KSP_SLOT_PROPERTY, in the window between NCryptOpenStorageProvider and the first operation: the slot is bound on first use rather than during initialisation, so the choice is still open. After binding it is refused with NTE_INVALID_HANDLE rather than accepted and ignored, because PKCS#11 cannot move a session between tokens and a caller told its selection succeeded would keep using the old one. A selection naming no present token is an error, never a fallback. Verified against TWO real tokens with different labels, PINs and keys; a test with one token cannot distinguish a working selection from an ignored one.	Vendor-specific	A	Covered	—	—
IFACE-05	**NotifyChangeKey**	Key-change notification handle	Microsoft KSP	A	Partial	Was a no-op returning ERROR_SUCCESS with the caller's HANDLE never written, so an application registering for notification was told it had succeeded and then waited on an uninitialised handle. It now validates the provider and the flags, clears *phEvent and refuses registration with NTE_NOT_SUPPORTED; unregistering succeeds because there is nothing to tear down. Real support needs a key-change channel a PKCS#11 token does not have - there is no C_WaitForSlotEvent equivalent for objects, and polling C_FindObjects on a timer would be a fabrication rather than a notification.	L
IFACE-06	**GetOperationProperty**	Async / overlapped operation state	Microsoft KSP	B	Not covered	Returns NTE_NOT_SUPPORTED. Only needed for overlapped I/O; PKCS#11 is synchronous so there is nothing to report.	L
IFACE-07	**PromptUser**	Interactive PIN / consent dialog	Smart card KSPs	B	Not covered	Returns NTE_NOT_SUPPORTED. Would need a UI thread and a window handle from NCRYPT_■WINDOW_■HANDLE_■PROPERTY.	L
IFACE-08	**FreeBuffer / FreeObject**	Provider-allocated memory release	All providers	A	Covered	—	—
IFACE-09	**VerifySignature**	NCryptVerifySignature through the provider	All providers implement the slot	A	Covered	—	—
TABLE-01	**Function table layout matches the SDK**	NCRYPT_■KEY_■STORAGE_■FUNCTION_■TABLE slot order and membership	All providers	A	Partial	The table uses designated initialisers, so the SDK header assigns slots by name and a wrong NAME is a build error. Since session 14 eight provider-scoped slots are also signature-checked: their KSP prototype equals the documented public NCrypt one, so test_■function_■table.c assigns each to a pointer type copied from and the build promotes incompatible-pointer-types to an error. That caught KSP_■NotifyChangeKey taking an NCRYPT_KEY_HANDLE where the parameter is HANDLE *phEvent. Key-scoped slots cannot be checked this way - NCryptSignHash takes (hKey, ...) while the slot takes (hProvider, hKey, ...) - so full confirmation still needs ncrypt_provider.h.	S

ID	Feature	Detail	Market support	Ev	Status	Gap — what closing it would take	Eff
BUILD-01	**Builds for Windows at all**	The shipped artefact is a Windows DLL	All providers	A	Partial	Phase 0 fixed all six root causes. MSVC compiles nine of the ten source files clean at /W3 /WX and links test_p11_layer. ksp_main.c cannot be built in hosted CI: ncrypt_provider.h does not exist on a windows-latest runner, established by a dir /s /b across C: and D: that returned only this repo's own test mock, after a WDK install that reported success. Needs a machine that has the header.	M
BUILD-02	**Continuous integration**	Automated build and test on the target platform	All providers	A	Covered	—	—
ASYMMETRIC RSA							
RSA-01	**RSA key generation**	2048 / 3072 / 4096 bits	All providers	A	Covered	—	—
RSA-02	**RSA 1024 and below**	512-1024 bits	MS Software KSP; Utimaco	B	Partial	Build-time opt-in only. KSP_RSA_MIN_BITS defaults to 2048; a legacy build can lower it, but the shipped default still rejects <2048 with NTE_BAD_LEN. Deliberate: silently allowing 1024 would weaken every deployment.	S
RSA-03	**RSA above 4096**	8192 / 16384 bits	MS Software KSP; Utimaco	B	Covered	—	—
RSA-04	**RSA arbitrary size increments**	64-bit (MS) or 8-bit (Utimaco) steps	MS Software KSP; Utimaco	B	Covered	—	—
RSA-05	**Custom public exponent**	Exponent other than 65537	Vendor-specific	C	Covered	—	—
RSA-06	**PKCS#1 v1.5 signing**	SHA-1 / 256 / 384 / 512	All providers	A	Covered	—	—
RSA-07	**PSS signing**	SHA-1 / 256 / 384 / 512 with caller salt length	All providers	B	Covered	—	—
RSA-08	**PSS with SHA-224**	SHA-224 salt and MGF1	Unknown	C	Covered	—	—
RSA-09	**PKCS#1 v1.5 decryption**	AT_KEYEXCHANGE keys	All providers	A	Covered	—	—
RSA-10	**OAEP decryption**	SHA-1 / 224 / 256 / 384 / 512	All providers	A	Covered	—	—
RSA-11	**OAEP application label**	pbLabel / cbLabel passed through	Unknown	C	Covered	—	—
RSA-12	**Raw RSA (no padding)**	CKM_RSA_X_509	Some HSMs	A	Covered	—	—
ASYMMETRIC ECDSA							
ECDSA-01	**ECDSA P-256**	secp256r1; 64-byte signature	All providers	A	Covered	—	—
ECDSA-02	**ECDSA P-384**	secp384r1; 96-byte signature	All providers	A	Covered	—	—
ECDSA-03	**ECDSA P-521**	secp521r1; 132-byte signature	All providers	A	Covered	—	—
ECDSA-04	**Brainpool curves**	brainpoolP256r1 / 384r1 / 512r1	Entrust; Utimaco (device)	C	Covered	—	—
ECDSA-05	**secp256k1**	Bitcoin curve	Entrust (device)	C	Covered	—	—
ECDSA-06	**DER to r||s conversion**	PKCS#11 returns DER; CNG wants raw	All providers	A	Covered	—	—
KEY AGREEMENT							
ECDH-01	**ECDH P-256 / P-384 / P-521**	CKM_ECDH1_DERIVE with CKD_NULL	MS Software KSP; Utimaco	B	Covered	—	—
ECDH-02	**SecretAgreement / DeriveKey / FreeSecret**	NCRYPT_■SECRET_■HANDLE lifecycle	MS Software KSP; Utimaco	B	Covered	—	—
ECDH-03	**BCRYPT_KDF_RAW_SECRET**	Raw Z value returned	MS Software KSP	B	Covered	—	—
ECDH-04	**BCRYPT_KDF_HASH**	SP 800-56A concatenation KDF	MS Software KSP	B	Covered	—	—
ECDH-05	**BCRYPT_KDF_HMAC / TLS_PRF / HKDF**	Protocol-specific KDFs	MS Software KSP	B	Covered	—	—

ID	Feature	Detail	Market support	Ev	Status	Gap — what closing it would take	Eff
ECDH-06	**Finite-field Diffie-Hellman**	DH 512-4096	MS Software KSP	B	Not covered	Out of scope by choice. ECDH supersedes it and no surveyed HSM KSP exposes it. Would need CKM_■DH_■PKCS_■KEY_■PAIR_■GEN plus CKM_■DH_■PKCS_■DERIVE.	L
ASYMMETRIC EDDSA							
EDDSA-01	**Ed25519 signing**	64-byte raw signature	None	A	Partial	Works against SoftHSM2 AND verified end to end against Kryoptic (session 10), but uses the non-standard identifier EDDSA_ED25519. CNG defines no EdDSA algorithm ID so no stock application can request it. Nothing to fix in the KSP; blocked on Microsoft.	N/A
EDDSA-02	**Ed448 signing**	114-byte raw signature	None	A	Partial	Same constraint as EDDSA-01. Note that Ed448 signing did not in fact work until session 10: CKM_EDDSA requires CK_EDDSA_PARAMS for Ed448 and the provider sent none. SoftHSM2 tolerates its absence, so this was invisible until a second token refused it.	N/A
EDDSA-03	**X25519 key agreement**	Curve25519 ECDH	MS CNG has BCRYPT_ECC_CURVE_25519	A	Covered	—	—
ASYMMETRIC DSA							
DSA-01	**DSA signing**	512-1024 bits	MS Software KSP	B	Not covered	Deliberately out of scope. Deprecated in modern Windows PKI and absent from every HSM KSP surveyed.	L
POST-QUANTUM							
PQC-01	**ML-DSA (Dilithium)**	FIPS 204	Microsoft Software KSP (GA late 2025; AD CS ML-DSA enrolment May 2026). Thales Luna firmware 7.9+ and Entrust nShield XC/5/nSaaS ship it in the device; whether their CNG providers expose it is not established	A	Partial	ML-DSA is wired through the KSP and gated on the capability probe: mechanism resolution, CKA_PARAMETER_SET key generation for all three parameter sets, key reopen, and signing. Verified end to end in session 10 against Kryoptic built with its pqc feature over OpenSSL 3.5.8 — generation, reopen by parameter set, and signature at the exact size for ML-DSA-44/65/87. Unreachable on SoftHSM2 2.7.0, which defines the mechanisms and implements none of them. Still Partial for one reason only: public key export is refused, because the CNG post-quantum key blob layout is in a Windows SDK header not available here. That is the same blocker as BUILD-01 and no backend fixes it.	M
PQC-02	**ML-KEM (Kyber)**	FIPS 203	Microsoft CNG GA (BCrypt side). Thales and Entrust device firmware. No KSP-side exposure established for any provider	B	Not covered	NOT the same blocker as PQC-01. The probe recognises CKM_ML_KEM, so the backend is not what stops this: encapsulation and decapsulation have no slot in the NCRYPT_■KEY_■STORAGE_■FUNCTION_■TABLE this provider implements, so a KSP can hold an ML-KEM key with no entry point to use it. Whether a newer ncrypt_provider.h adds such slots cannot be determined here — same missing header as BUILD-01.	L
PQC-03	**LMS / XMSS**	Stateful hash-based signatures	Thales Luna, Entrust nShield (device claims only)	C	Not covered	Blocked at the backend, unlike PQC-01: no PKCS#11 v3.2 token surveyed implements LMS or XMSS. Stateful schemes also need one-time-key state persistence the KSP has no model for, and a lost state update forges signatures.	L
SYMMETRIC AES							
AES-01	**AES key generation**	128 / 192 / 256 bits	None via KSP	A	Covered	—	—
AES-02	**AES-ECB**	No IV	None via KSP	A	Covered	—	—
AES-03	**AES-CBC**	16-byte IV	None via KSP	A	Covered	—	—
AES-04	**AES-CBC with padding**	NCRYPT_■PAD_■CIPHER_■FLAG	None via KSP	A	Covered	—	—
AES-05	**AES-GCM**	12-byte nonce; 128-bit tag; AAD	None via KSP	A	Covered	—	—
AES-06	**AES-CTR**	16-byte counter block	None via KSP	A	Partial	Works but ChainingModeCTR is a KSP extension. CNG defines no standard counter-mode string so no stock application will select it.	N/A
AES-07	**AES-CCM**	Authenticated mode	None via KSP	A	Covered	—	—

ID	Feature	Detail	Market support	Ev	Status	Gap — what closing it would take	Eff
AES-08	**AES-CFB**	Cipher feedback	None via KSP	A	Partial	Implemented and verified against a live token for both feedback sizes CNG can reach: the 8-bit default and full-block via MessageBlockLength. Partial because CNG permits any feedback size up to the block, while PKCS#11 defines mechanisms only for 1, 8, 64 and 128 bits; a size with no mechanism is refused rather than silently rounded to a different cipher.	M
AES-09	**AES key wrap**	RFC 3394 / 5649	Some HSMs (device)	A	Covered	—	—
AES-10	**AES-CMAC**	Block-cipher MAC	Some HSMs (device)	A	Covered	—	—
SYMMETRIC MAC							
HMAC-01	**HMAC-SHA1 / 256 / 384 / 512**	Generic secret keys	None via KSP	A	Partial	Works against SoftHSM2 but HMAC_SHA* are our own identifiers. CNG does HMAC through BCrypt algorithm handles not KSP key algorithms.	N/A
HMAC-02	**HMAC-SHA224**	SHA-224 variant	None	C	Covered	—	—
SYMMETRIC LEGACY							
3DES-01	**3DES / DES**	56 / 112 / 168 bits	None via KSP	B	Not covered	Deliberately out of scope. Deprecated and excluded from modern HLK requirements despite SoftHSM2 supporting 14 DES mechanisms.	L
KEY FORMATS							
FMT-01	**BCRYPT_RSAPUBLIC_BLOB export**	Header + exponent + modulus	All providers	A	Covered	—	—
FMT-02	**BCRYPT_ECCPUBLIC_BLOB export**	Header + X + Y	All providers	A	Covered	—	—
FMT-03	**EdDSA public blob export**	Generic ECC magic + raw point	None	A	Covered	—	—
FMT-04	**BCRYPT_RSAPUBLIC_BLOB import**	Creates a session public key object	Vendor-specific	C	Covered	—	—
FMT-05	**BCRYPT_ECCPUBLIC_BLOB import**	Creates a session public key object	Vendor-specific	C	Covered	—	—
FMT-06	**Private key export**	RSAFULLPRIVATE / ECCPRIVATE	None (all refuse)	A	Not covered	Refused by design with NTE_NOT_SUPPORTED. Keys are CKA_■EXTRACTABLE=FALSE. This is correct HSM behaviour not a gap.	N/A
FMT-07	**Private key import**	Wrapping an external private key onto the token	Some HSMs	C	Not covered	Returns NTE_NOT_SUPPORTED. Would need C_CreateObject with private material or C_UnwrapKey. Contradicts the HSM posture; only add if a migration workflow demands it.	L
FMT-08	**BCRYPT_KEY_DATA_BLOB**	Raw symmetric key material	MS Software KSP	A	Covered	—	—
FMT-09	**PKCS#8 blob**	NCRYPT_■PKCS8_■PRIVATE_■KEY_■BLOB	MS Software KSP	B	Not covered	Same constraint as FMT-07: it is a private-key format and keys are non-extractable.	L
FMT-10	**OPAQUETRANSPORT blob**	Vendor-opaque key transport	Vendor-specific	C	Not covered	Would let keys move between instances of the same provider. Needs a defined wrapping format; no standard to follow.	L
KEY PROPERTIES							
PROP-01	**NCRYPT_ALGORITHM_PROPERTY**	Read	All providers	A	Covered	—	—
PROP-02	**NCRYPT_LENGTH_PROPERTY**	Read and write before finalize	All providers	A	Covered	—	—
PROP-03	**NCRYPT_KEY_TYPE_PROPERTY**	AT_SIGNATURE / AT_KEYEXCHANGE	All providers	A	Covered	—	—
PROP-04	**NCRYPT_NAME / UNIQUE_NAME**	Key label	All providers	A	Covered	—	—

ID	Feature	Detail	Market support	Ev	Status	Gap — what closing it would take	Eff
PROP-05	**NCRYPT_EXPORT_POLICY_PROPERTY**	Always 0 (non-exportable)	All providers	A	Covered	—	—
PROP-06	**NCRYPT_KEY_USAGE_PROPERTY**	Signing / decrypt / key agreement flags	All providers	A	Covered	—	—
PROP-07	**NCRYPT_ALGORITHM_GROUP_PROPERTY**	RSA / ECDSA / ECDH / EDDSA / AES / HMAC	All providers	A	Covered	—	—
PROP-08	**NCRYPT_CHAINING_MODE_PROPERTY**	Read and write on symmetric keys	MS Software KSP (BCrypt)	A	Covered	—	—
PROP-09	**NCRYPT_INITIALIZATION_VECTOR**	Read and write on symmetric keys	MS Software KSP (BCrypt)	A	Covered	—	—
PROP-10	**NCRYPT_BLOCK_LENGTH_PROPERTY**	AES block size	MS Software KSP	B	Covered	—	—
PROP-11	**NCRYPT_SECURITY_DESCR_PROPERTY**	Per-key ACL	MS Software KSP	B	Not covered	Returns NTE_NOT_SUPPORTED. PKCS#11 has no ACL model; would need a side-channel store mapping key labels to SDDL strings.	L
PROP-12	**NCRYPT_UI_POLICY_PROPERTY**	Strong-key protection prompt	MS Software KSP; smart cards	B	Not covered	Depends on PromptUser (IFACE-07). Same UI-thread requirement.	L
PROP-13	**NCRYPT_PIN_PROPERTY**	Per-key PIN	Smart card KSPs	A	Covered	—	—
PROP-14	**NCRYPT_CERTIFICATE_PROPERTY**	Certificate stored alongside the key	Smart card KSPs	A	Covered	—	—
PROP-15	**NCRYPT_WINDOW_HANDLE_PROPERTY**	Parent HWND for dialogs	Smart card KSPs	B	Not covered	Only meaningful with PromptUser (IFACE-07).	L
PROP-16	**NCRYPT_USE_COUNT_PROPERTY**	Key usage counter	Some HSMs	C	Not covered	Would need a counter object on the token incremented per operation. SoftHSM2 has no atomic counter primitive.	L
PROP-18	**NCRYPT_EXPORT_POLICY_PROPERTY**	Caller declares at creation whether a key may leave the token	MS Software KSP; every provider that supports key backup	A	Covered	—	—
PROP-19	**NCRYPT_LENGTHS_PROPERTY**	Supported key sizes for the algorithm (NCRYPT_■SUPPORTED_■LENGTHS)	All providers	A	Covered	—	—
KEY LIFECYCLE							
LIFE-01	**CreatePersistedKey**	Immediate and deferred generation	All providers	A	Covered	—	—
LIFE-02	**FinalizeKey**	Deferred generation trigger	All providers	A	Covered	—	—
LIFE-03	**OpenKey by name**	Asymmetric and symmetric	All providers	A	Covered	—	—
LIFE-04	**DeleteKey**	Destroys all token objects	All providers	A	Covered	—	—
LIFE-05	**EnumKeys**	Paged enumeration with state	All providers	A	Covered	—	—
LIFE-06	**EnumAlgorithms**	NCryptEnumAlgorithms support	All providers	B	Covered	—	—
LIFE-07	**IsAlgSupported**	NCryptIsAlgSupported	All providers	B	Covered	—	—

ID	Feature	Detail	Market support	Ev	Status	Gap — what closing it would take	Eff
LIFE-08	**Machine vs user key scope**	NCRYPT_■MACHINE_■KEY_■FLAG is honoured on CreatePersistedKey, OpenKey and EnumKeys. Within one token it is a CKA_LABEL prefix ("m/name" or "u/name"), which keeps distinct keys distinct and is NAMESPACING, NOT ISOLATION. Set KSP_■MACHINE_■TOKEN_■LABEL and KSP_■USER_■TOKEN_■LABEL (or the slot equivalents) and the flag instead selects WHICH TOKEN, each with its own PIN and its own session pool, so isolation is as strong as the token boundary PKCS#11 does have: a caller holding the user PIN cannot read machine keys because it cannot log in to read them. Setting only one of a pair, or two names resolving to one token, is refused rather than half-applied — isolation that reports success and does not isolate is worse than none. Unconfigured behaviour is unchanged. Verified against two real tokens with different PINs, including a case run WITHOUT the machine PIN in which the machine token must be unreachable.	All providers	A	Covered	—	—
OPERATIONAL							
OPS-01	**Hardware key protection**	Tamper-resistant storage	All commercial providers	A	Not covered	Fundamental: SoftHSM2 is a software token using encrypted SQLite. Only fixable by pointing the KSP at a real HSM which the PKCS#11 abstraction already allows.	N/A
OPS-02	**FIPS 140-2/3 validation**	Certified cryptographic module	All commercial providers	B	Not covered	Follows from OPS-01. Requires certified hardware and a formal validation programme.	N/A
OPS-03	**Authenticode-signed binary**	Commercial code-signing certificate	All commercial providers	A	Not covered	Signing is fully tooled: tools/sign_ksp.ps1 signs, timestamps and verifies, taking a certificate from the Windows store, a PFX (password from the environment, never a parameter) or Azure Trusted Signing, and the CI windows job signs automatically once the KSP_■SIGN_■PFX_■BASE64 secret exists. What remains is not engineering: an OV or EV code-signing certificate must be bought by a verified legal entity. Nothing in this repository can supply that, so the row stays Not covered — the binary is unsigned.	N/A
OPS-04	**Multiple slots / tokens**	Selecting among several tokens	All commercial providers	B	Covered	—	—
OPS-05	**Key attestation**	TPM / HSM-signed key origin proof	MS Platform Crypto; some HSMs	B	Not covered	No PKCS#11 standard mechanism exists. Vendor-specific and unreachable through a portable PKCS#11 layer.	L
OPS-06	**Load balancing / HA**	Multiple HSM members	Thales; Entrust	C	Not covered	Handled inside vendor PKCS#11 libraries. Our KSP would inherit it automatically when pointed at such a library; nothing to implement.	N/A
OPS-07	**Session pool concurrency**	Concurrent NCrypt calls	All providers	A	Covered	—	—
OPS-08	**Re-initialisation without restart**	Recover from a bad DLL path	Vendor-specific	A	Covered	—	—
OPS-09	**PIN caching / re-login**	Session recovery after token logout	All commercial providers	C	Covered	—	—

ID	Feature	Detail	Market support	Ev	Status	Gap — what closing it would take	Eff
OPS-10	**Logging / diagnostics**	Trace output for support	All providers	A	**Covered**	—	—
OPS-13	**Runtime capability discovery**	Advertised algorithms reflect what the token implements, not a compiled-in list	All commercial providers (implicit — each ships for its own device)	A	**Covered**	—	—
OPS-14	**Backend portability**	Any PKCS#11 v2.40+ module can back the provider (KSP_PKCS11_LIB)	Not offered — commercial providers are bound to their own hardware	A	**Covered**	—	—

Gap analysis

30 rows are open. 19 are blocked outside this repository — a CNG identifier Microsoft has not defined, a PKCS#11 mechanism that does not exist, a header nobody here has, hardware, or a purchase. The remaining **11 are decisions this project made**, and those are the only ones a reader can argue with.

Why that split is the one worth printing. Three rows were recorded as deliberate positions and have since been closed: IFACE-04 (token selection), LIFE-08 (machine/user isolation) and FMT-08 (symmetric key export). The error was identical each time — the "position" rested on how the provider happened to work at the time, not on anything in CNG or PKCS#11. The slot was bound during initialisation, so selection looked impossible; scoping was a label prefix, so isolation looked impossible; keys were non-extractable by default, so export looked refused. A claim that something cannot be done has to rest on something outside this code, or it is a description masquerading as a constraint.

Waiting on ncrypt_provider.h and a Windows machine (3)

ID	Feature	St	What works today	What the blocker rests on
BUILD-01	**Builds for Windows at all**	P	MSVC compiles nine of the ten source files clean at /W3 /WX and links test_■p11_■layer.exe. The mingw cross-compile is clean on all ten minus ksp_main.c.	ncrypt_provider.h, which declares NCRYPT_■KEY_■STORAGE_■FUNCTION_■TABLE. A dir /s /b across C: and D: on a windows-latest runner returned only this repository's own test mock, after a WDK install that reported success. It is not in the Windows SDK.
PQC-01	**ML-DSA (Dilithium)**	P	ML-DSA generates, reopens by parameter set and signs at all three sets, verified against Kryoptic built with its pqc feature over OpenSSL 3.5.8. Gated on the capability probe, so it is dark on SoftHSM2 and live on a v3.2 token.	The CNG post-quantum key blob layout, which is in a Windows SDK header not available here. Public key export is the only thing missing. A guessed layout would pass every test here and fail only on Windows — the BCRYPT_■SHA224_■ALGORITHM mistake exactly.
TABLE-01	**Function table layout matches the SDK**	P	Designated initialisers make a wrong slot NAME a build error. Eight provider-scoped slots are also signature-checked against , which is what caught KSP_■NotifyChangeKey holding an NCRYPT_KEY_HANDLE where the parameter is HANDLE *phEvent.	The same missing header. Key-scoped slots cannot be checked against the public prototype, because it is not the slot's shape: NCryptSignHash takes (hKey, ...) while the slot takes (hProvider, hKey, ...). Asserting the public shape there would be inventing a fact.

Waiting on Microsoft — CNG defines no identifier or no slot (5)

ID	Feature	St	What works today	What the blocker rests on
AES-06	**AES-CTR**	P	AES-CTR encrypts and decrypts, verified against a live token, selected through NCRYPT_■CHAINING_■MODE_■PROPERTY.	CNG defines no counter-mode chaining string, so ChainingModeCTR is a provider extension that no stock application will select.
EDDSA-01	**Ed25519 signing**	P	Ed25519 signs, and is verified end to end against Kryoptic. The absent CK_EDDSA_PARAMS that selects the pure context-free form is deliberate and fault-injected.	CNG defines no EdDSA algorithm identifier at all, so EDDSA_ED25519 is this provider's own string and no stock application will ask for it. Nothing in the KSP can change that.
EDDSA-02	**Ed448 signing**	P	Ed448 signs against Kryoptic with CK_EDDSA_PARAMS supplied, which Ed448 requires and Ed25519 must not have. The asymmetry is deliberate and both mistakes are fault-injected. It did NOT work before session 10: SoftHSM2 tolerates the missing parameter, so a second token was needed to see it.	Same as EDDSA-01 — no CNG identifier exists.
HMAC-01	**HMAC-SHA1 / 256 / 384 / 512**	P	HMAC signing works over CKM_SHA*_HMAC for SHA-1 through SHA-512, verified against a live token. It had never actually worked before session 10 — KSP_SignHash passed hPrivKey unconditionally and secret keys live in hSecretKey.	CNG reaches HMAC through BCrypt algorithm handles, not through KSP key algorithms, so HMAC_SHA* are this provider's own identifiers.
PQC-02	**ML-KEM (Kyber)**	No	The probe recognises CKM_ML_KEM, so the backend is not the obstacle. Nothing is advertised, deliberately.	The function table has no slot for encapsulation or decapsulation, so a KSP could hold an ML-KEM key with no entry point to use it. Whether a newer ncrypt_provider.h adds such slots cannot be established here — the same missing header.

Waiting on PKCS#11 — no mechanism exists to build on (7)

ID	Feature	St	What works today	What the blocker rests on
FMT-10	**OPAQUETRANSPORT blob**	No	Nothing. Not advertised.	OPAQUETRANSPORT has no defined wrapping format to follow, so there is no standard to implement against — only a format this provider would invent and no one else could read.
IFACE-05	**NotifyChangeKey**	P	Validates the provider and flags, clears *phEvent and refuses registration with NTE_NOT_SUPPORTED. Unregistering succeeds, because there is nothing to tear down. Before session 14 it returned ERROR_SUCCESS without writing the handle, so a caller waited on an uninitialised HANDLE.	PKCS#11 has no object-change channel: there is no C_WaitForSlotEvent equivalent for objects. Polling C_FindObjects on a timer would be a fabrication dressed as a notification. THIS IS THE ROW CLOSEST IN SHAPE TO the three that were wrongly called deliberate — it would become actionable the moment a backend offered such a channel, so it is worth re-reading rather than inheriting.

ID	Feature	St	What works today	What the blocker rests on
IFACE-06	**GetOperationProperty**	No	Refused with NTE_NOT_SUPPORTED.	The property reports on an asynchronous operation and PKCS#11 is synchronous, so there is no operation state to report however much code is written. This row sat in the deliberate column until the pairing check was run against its own text, which already said "a fact about PKCS#11 rather than about this code" — so it was mislabelled by the person who wrote the reason, and the cross-check is what caught it.
OPS-05	**Key attestation**	No	Nothing. Not advertised.	Key attestation has no standard PKCS#11 mechanism. It is vendor-specific, so it is unreachable through a portable PKCS#11 layer by construction.
PQC-03	**LMS / XMSS**	No	Nothing. Not advertised.	No PKCS#11 v3.2 token surveyed implements LMS or XMSS, so unlike PQC-01 this is blocked at the backend. The stateful schemes also need one-time-key state persistence the KSP has no model for, and a lost state update forges signatures — a correctness hazard, not just missing code.
PROP-11	**NCRYPT_SECURITY_DESCR_PROPERTY**	No	Refused with NTE_NOT_SUPPORTED rather than accepted and ignored.	PKCS#11 has no ACL model, so there is nothing to map an SDDL string onto. A side store from key label to descriptor would be this provider inventing an access control system it cannot enforce.
PROP-16	**NCRYPT_USE_COUNT_PROPERTY**	No	Refused with NTE_NOT_SUPPORTED.	No atomic counter primitive in PKCS#11 or SoftHSM2. A read-modify-write on an object attribute is not a use count under concurrency, and a wrong count is worse than no count.

Waiting on hardware, a certificate purchase, or a validation programme (4)

ID	Feature	St	What works today	What the blocker rests on
OPS-01	**Hardware key protection**	No	Nothing, and it cannot: SoftHSM2 is a software token over encrypted SQLite. The provider reports NCRYPT_■IMPL_■SOFTWARE_■FLAG, truthfully — it used to say hardware in the source while emitting the software value.	Hardware. The PKCS#11 abstraction already allows pointing the KSP at a real HSM, so this closes by deployment rather than by code.
OPS-02	**FIPS 140-2/3 validation**	No	Nothing.	Certified hardware and a formal validation programme. Follows from OPS-01 and is not an engineering task.
OPS-03	**Authenticode-signed binary**	No	The signing pipeline is complete and self-tested: tools/sign_ksp.ps1 signs, timestamps and verifies, taking a certificate from the Windows store, a PFX whose password comes from the environment and never a parameter, or Azure Trusted Signing. CI signs automatically once the secret exists.	An OV or EV code-signing certificate, which must be bought by a verified legal entity. Nothing in this repository can supply that, so the shipped binary is unsigned and the row stays Not covered.
OPS-06	**Load balancing / HA**	No	Nothing here, and nothing needs to: it is handled inside vendor PKCS#11 libraries.	A vendor library. The provider inherits load balancing and failover automatically when pointed at one, so there is nothing to implement.

Decisions this project made — revisitable by definition (11)

ID	Feature	St	What works today	What the blocker rests on
3DES-01	**3DES / DES**	No	Nothing, although SoftHSM2 implements fourteen DES mechanisms, so the backend is not the obstacle.	Scope and age. Excluded from modern HLK requirements. Implementable at any time; deliberately not implemented.
AES-08	**AES-CFB**	P	Both feedback sizes CNG can actually reach: the 8-bit default, and full-block through BCRYPT_■MESSAGE_■BLOCK_■LENGTH. CKM_AES_CFB64 is mapped for a MessageBlockLength of 8. All gated on the capability probe and verified against a live token, which asserts the two sizes produce DIFFERENT ciphertext — a provider ignoring the property cannot pass.	Two things, and only one of them is ours. The ASYMMETRY is external: CNG permits any feedback size up to the block, PKCS#11 defines mechanisms for four of them, and nothing here can add the missing ones. What is DELIBERATE is the handling — a size with no mechanism is refused rather than rounded to the nearest, because rounding would silently select a different cipher and produce ciphertext nothing else could decrypt. The row is classified as our decision because that refusal is the part a reader might want to argue with.
DSA-01	**DSA signing**	No	Nothing.	Scope. Deprecated in modern Windows PKI and absent from every HSM KSP surveyed.
ECDH-06	**Finite-field Diffie-Hellman**	No	Nothing. ECDH on nine curves covers the agreement surface.	Scope. CKM_■DH_■PKCS_■KEY_■PAIR_■GEN and CKM_DH_PKCS_DERIVE both exist, so this IS implementable — it is excluded because ECDH supersedes it and no surveyed HSM KSP exposes it. Of all the deliberate rows this is the one with the clearest path if a deployment ever needed it.

ID	Feature	St	What works today	What the blocker rests on
FMT-06	**Private key export**	No	Public key export in CNG blob format for every supported family. Private key export is refused with NTE_NOT_SUPPORTED.	The HSM posture, enforced on the token rather than in the provider: keys are CKA_■EXTRACTABLE=FALSE unless NCRYPT_■EXPORT_■POLICY_■PROPERTY relaxes it, and even then only wrapped export opens. This is correct behaviour, not an unfinished feature.
FMT-07	**Private key import**	No	Public key import creates a real CKO_PUBLIC_KEY; truncated and unknown-curve blobs are rejected rather than silently accepted. Symmetric key unwrap works and creates the key sensitive and non-extractable, so wrapping is a way IN and not a way back out.	The HSM posture again. C_CreateObject with private material or C_UnwrapKey would both work, so this is a choice: a key the provider did not generate has an unknown history. Worth revisiting only if a migration workflow demands it.
FMT-09	**PKCS#8 blob**	No	Nothing: it is a private-key format.	FMT-07, exactly. Same posture, same reconsideration trigger.
IFACE-07	**PromptUser**	No	Refused with NTE_NOT_SUPPORTED.	A UI thread and a window handle. A KSP is loaded into whatever process called NCrypt, including services with no desktop, so prompting is a deployment decision rather than a capability to add unconditionally.
PROP-12	**NCRYPT_UI_POLICY_PROPERTY**	No	Refused with NTE_NOT_SUPPORTED.	IFACE-07. A UI policy with no prompt to govern would be a stored value nothing reads.
PROP-15	**NCRYPT_WINDOW_HANDLE_PROPERTY**	No	Refused with NTE_NOT_SUPPORTED.	IFACE-07, for the same reason as PROP-12.
RSA-02	**RSA 1024 and below**	P	Any multiple of 64 from KSP_RSA_MIN_BITS to 16384. A legacy build can lower the floor; the shipped default refuses below 2048 with NTE_BAD_LEN.	A security choice, not a constraint: the mechanism exists and the code path works. Silently allowing 1024 would weaken every deployment that did not ask for it, so it is an opt-in at build time rather than a refusal.

Gaps outside this matrix's scope

Every row above is a capability observed across shipping CNG KSPs. A defect in *this* provider's own conformance surface is not such a capability, so no row here would ever point at one. That blind spot has cost three findings — two stubs found by reading the function table, and the first below found by making a test repeatable — so they are recorded rather than left in a commit message.

Gap	Detail	How it was found	Status
NCryptCreatePersistedKey does not return NTE_EXISTS	Nothing checks whether the name is already taken, so a second create adds a second object with the same CKA_LABEL and NCryptOpenKey returns whichever the token's search hands back first. Microsoft documents NTE_EXISTS for this case, with NCRYPT_■OVERWRITE_■KEY_■FLAG as the way to ask for replacement.	While making the two-token suite repeatable: two deletes both reported success and a key was still openable, because earlier runs had left duplicates behind.	Open. Not fixed as part of proposals A to D, because doing it properly means the overwrite flag as well as the check, with tests in both directions.
Key-scoped function table slots are unverifiable	The mock types every slot as void *, so a wrong parameter TYPE is not a build error on Linux, and MSVC cannot compile ksp_main.c without ncrypt_provider.h. Eight provider-scoped slots are now checked against ; key-scoped ones cannot be, because the public prototype is not the slot's shape.	Session 14, by asking why KSP_■NotifyChangeKey had survived with the wrong parameter type.	Tracked as TABLE-01, which is the one matrix row that does cover a conformance question rather than a capability.

Maintaining this document

The source of truth is `softhsm_ksp/docs/feature-matrix.csv`. One row per capability. To record that a gap has been closed, change that row's *status* to Covered and clear its *gap_solution*; to add a newly observed market feature, append a row with a fresh ID. Then regenerate:

```
python3 generate_feature_matrix.py
```

Every number on page 1 is computed from the CSV at generation time, so the summary cannot fall out of step with the table. Because the CSV is plain text, a change to coverage shows up as a one-line diff in review rather than an opaque binary change.

Startable here: 10 — 1 small, 1 medium, 8 large. These are the rows whose blocker is a decision this project made, so work could begin on any of them today.

Sized but blocked: 11 — 1 small, 2 medium, 8 large. Each has a remedy and an estimate, and none of them can start here: the blocker is a header nobody in this project has, a CNG identifier Microsoft has not defined, a PKCS#11 mechanism that does not exist, hardware, or a purchase. They are shown because what a row would take *if* unblocked is worth knowing, and hidden from the

startable count because reading it as backlog is exactly the mistake this document used to invite — the two numbers were one number until it was measured, and that number said 21 when 10 was true.

`Content fingerprint: a1989d4d8b33` — of the data this document was built from. `tests/check_generated_artifacts.py` recomputes it, so a matrix edited without regenerating is a CI failure rather than a document that quietly describes the previous version of itself.

A caution on the market columns. An HSM datasheet is not a statement about its CNG provider. AWS CloudHSM performs AES, DES3 and HMAC in hardware, yet its KSP answers NCryptIsAlgSupported for exactly four identifiers. Every grade-C cell here carries that uncertainty. See docs/11-market-comparison.md for the full audit and its sources.